ITPROFEL2: OPEN SOURCE PROGRAMMING

Learning Module: Finals - Week 4

Instructor: Henry V. Dela Cruz

Topic: Introduction to Object-Oriented Programming (OOP)

I. Overview and Objectives

Until now, our scripts have been procedural—a sequence of functions and commands executing top-to-bottom. As we prepare to build larger, enterprise-grade systems, we must transition to a more powerful paradigm: Object-Oriented Programming. This week, we learn how to map real-world entities into code.

Intended Learning Outcomes (ILOs):

- Define Object-Oriented Programming (OOP) and its core philosophy.
- Explain the importance of using classes in large-scale software programs.
- Differentiate between classes (blueprints) and functions (actions).
- Write the syntax and structure of a basic class definition in Python.

II. What is Object-Oriented Programming?

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects." Instead of just writing functions that operate on separate data variables, OOP bundles the **data** (attributes) and the **behavior** (methods) together into a single unit.

The Importance of Classes in Large-Scale Programs

In open-source software, hundreds of developers might work on the same codebase. OOP provides several massive advantages:

- **Modularity:** Code is organized into distinct classes, making it easier to troubleshoot and maintain.
- **Reusability:** Once a class is written, it can be instantiated hundreds of times or inherited by other classes without rewriting logic.
- **Scalability:** Complex systems (like a university database or a game engine) are much easier to conceptualize when broken down into interacting objects (e.g., `Student`, `Course`, `Instructor`).

III. Differences Between Classes and Functions

Feature	Functions (Procedural)	Classes (Object-Oriented)
Primary Focus	Action-oriented (doing something to data).	State-oriented (bundling data and actions together).
Data Handling	Takes data in as parameters, returns a result.	Stores its own data internally (attributes) and modifies it via its own functions (methods).
Analogy	A recipe (instructions to bake a cake).	A factory blueprint (creates an oven object that knows how to bake).

IV. Syntax and Structure of a Class Definition

In Python, we use the `class` keyword to define a blueprint. By convention, class names use **PascalCase** (e.g., `UserProfile`, `NetworkConnection`), unlike functions which use `snake_case`.

When designing a system's architecture, developers often create empty classes first using the `pass` keyword, filling in the logic later. Adding a *docstring* (triple quotes) is a standard practice to document what the class will eventually do.

```
# Defining a class blueprint
class ServerNode:
    "This class represents a server node. It holds IP address and status."
    pass  # The pass statement allows an empty block without syntax errors

# Creating an actual object (instantiation) from the blueprint
node_alpha = ServerNode()

# Verifying the object type
print(type(node_alpha))  # Output: <class '__main__.ServerNode'>
```

Laboratory Coding Session: System Architecture Blueprint

Scenario: You are tasked with designing the foundation of a new backend profile management system. Before adding variables or functions, you must outline the core entity using OOP principles. You will create a completely empty class skeleton and document its future purpose.

Instructions (Your code must implement these exact rules):

1. Use the `class` keyword to define a new class named `UserEntity`.
2. Inside the class, provide a docstring (`"..."`) that clearly states:
    - What this class represents.
    - Three pieces of data (attributes) it will eventually store.
3. Use the `pass` keyword so Python doesn't throw an error for having an empty class body.
4. Outside the class, print a system startup message.
5. Instantiate a new object from your `UserEntity` class and assign it to a variable named `test_user`.
6. Print the `type()` of your newly created `test_user` object to verify it was created successfully.

Sample Program Run (Expected Output):

```
--- SYSTEM ARCHITECTURE INITIALIZED ---
Verifying primary entity structure...

Object Type Confirmed: <class '__main__.UserEntity'>
Status: Blueprint ready for attributes.
```

Submit your completed `.py` *file to the instructor. Keep this file safe; we will be injecting data into this exact blueprint next week!*